

Apache Airflow 3

Apache airflow is an open-source to write, schedule and manage workflow as code. Whenever you have actions that depend on another and must be performed in specific order, you can define them as workflow in airflow.

Common Example:

A data pipeline that extracts data from various sources, combines it and loads to destination. With airflow, we can define the pipeline in python file and schedule it to run automatically.

So Apache airflow empowers you to automate the workflow using code. Airflow environment contains one or more dags. DAG - Directed Acyclic Graphs.

The term pipeline refers to one or more dags in an airflow environment plus their associated actions and data objects in other tools.

Although dags can define workflow managing infrastructure to fine-tuning generative models.

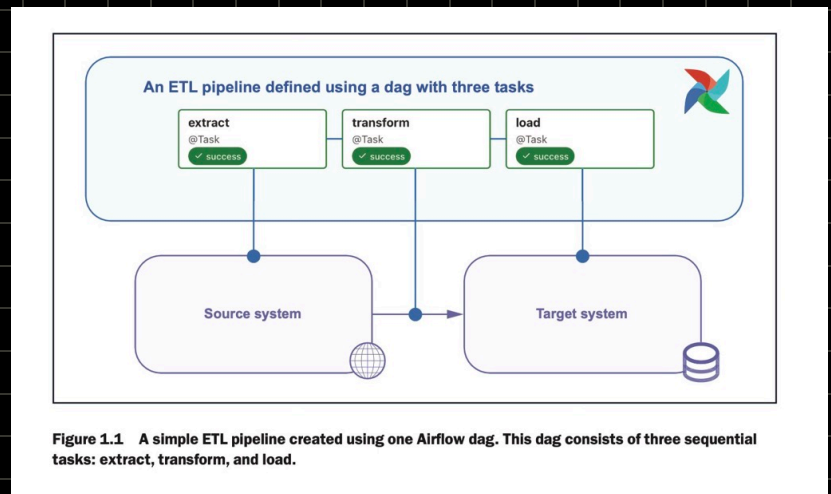
In DAG, the nodes represent "Task" and Edges denotes the "dependencies"

There are two approaches:

- (i) Task Oriented Approach
- (ii) Asset oriented approach.

Task Oriented Approach:

A common way to create a dag is over flow is to create individual tasks, each of which performs an action in the pipeline, then define their dependencies.



Code Structure:

from airflow.sdx import chain, dag, task
@dag → defines dag with default parameters

`def my_pipeline():` → dag decorated function name is ID of dag

`@task` → defines task with default parameters.

`def extract():` → task decorated python function is
pass ID of the task.

`@task`

`def transform():`
pass

`@task`

`def load():`
pass

`chain(extract(), transform(), load())`

`my_pipeline()`

cii) Asset Oriented Approach:

Assets are objects within airflow that represent real or abstract data structures in other systems.

You can write pipeline using assets that depends on another instead of tasks. This way of writing pipeline is called asset oriented approach.

Code Structure:

`from airflow.sdk import asset`

`@asset(schedule="daily")`

```
def extracted_data():
```

```
    pass
```

```
@assert(schedule = [extracted_data])
```

```
def transformed_data():
```

```
    pass
```

```
@assert(schedule = [transformed_data])
```

```
    pass
```